

PARCEL: Package Arrival, Reception Coordination and Entry Logger

Final Project Report/Summary

Andrew Kesler, Kina Li, Kyler Jeong (Anwar FanClub)

→ [Full Video Demonstration Link!](#) ←

Image of all Group Members with the Project:



Problem Statement:

Package theft and delivery mishandling are growing concerns, especially for individuals who frequently receive packages while away from home. Many deliveries are left unattended on porches, making them easy targets for theft. Additionally, packages may be damaged due to improper handling, and recipients often lack a reliable way to confirm their arrival without physically checking.

Existing solutions, such as smart doorbells and lockboxes, offer partial solutions that require constant user monitoring or are costly and not widely accessible. We hope to create an automated, secure, and user-friendly solution, that can lock packages upon delivery, monitor disturbances, and notify the recipient in real time.

Proposed Solution

Our project aims to develop a Smart Package Reception and Security System using three ESP32 microcontrollers, sensors, and actuators to provide a secure, automated, and interactive package storage solution. The system consists of a lockable, laser-cut wood package box that:

- Detects package placement via a force sensor.
- Locks the package securely using a motorized latch.
- Monitors disturbances such as shaking or unauthorized attempts to open the box using an inertial sensor.
- Tracks activity around the box using an infrared sensor.
- Notifies the recipient of a package delivery or potential security breach via a remote buzzer alert and an email notification.
- Allows remote unlocking via a handheld remote carried by the recipient.

This system aims to ensure that packages remain safe until retrieved by the intended recipient while providing real-time tracking and security monitoring.

ESP32 Module	Function	Sensors & Actuators
ESP 1 (Inside Box - Package Security & Monitoring)	Detects package, locks/unlocks box	Force sensor (detects package placement), Motorized latch (locks/unlocks the package).
ESP 2 (External Monitoring - Theft Prevention & Alerts)	Monitors surroundings and detects disturbances; sends email alerts	Distance sensor (monitors movement near the box), Inertial sensor (detects movement or tampering), email notifications

ESP32 1 CODE (Typed Out):***HX711.py***

```

from utime import sleep_us, time
from machine import Pin
from micropython import const

```

```

class HX711Exception(Exception):
    pass

```

```

class InvalidMode(HX711Exception):
    pass

```

```

class DevicesNotReady(HX711Exception):
    pass

```

```

class HX711(object):
    """
    Micropython driver for Avia Semiconductor's HX711
    24-Bit Analog-to-Digital Converter
    """
    CHANNEL_A_128 = const(1)
    CHANNEL_A_64 = const(3)
    CHANNEL_B_32 = const(2)

    DATA_BITS = const(24)
    MAX_VALUE = const(0x7fffff)
    MIN_VALUE = const(0x800000)
    READY_TIMEOUT_SEC = const(5)
    SLEEP_DELAY_USEC = const(80)

    def __init__(self, d_out: int, pd_sck: int, channel: int = CHANNEL_A_128):

```

```

self.d_out_pin = Pin(d_out, Pin.IN)
self.pd_sck_pin = Pin(pd_sck, Pin.OUT, value=0)
self.channel = channel

```

```

def __repr__(self):
    return "HX711 on channel %s, gain=%s" % self.channel

```

```

def _convert_from_twos_complement(self, value: int) -> int:
    """
    Converts a given integer from the two's complement format.
    """
    if value & (1 << (self.DATA_BITS - 1)):
        value -= 1 << self.DATA_BITS
    return value

```

```

def _set_channel(self):
    """
    Input and gain selection is controlled by the
    number of the input PD_SCK pulses
    3 pulses for Channel A with gain 64
    2 pulses for Channel B with gain 32
    1 pulse for Channel A with gain 128
    """
    for i in range(self._channel):
        self.pd_sck_pin.value(1)
        self.pd_sck_pin.value(0)

```

```

def _wait(self):
    """
    If the HX711 is not ready within READY_TIMEOUT_SEC
    the DevicesNotReady exception will be thrown.
    """
    t0 = time()
    while not self.is_ready():
        if time() - t0 > self.READY_TIMEOUT_SEC:
            raise DevicesNotReady()

```

```

@property

```

```

def channel(self) -> tuple:
    """
    Get current input channel in a form
    of a tuple (Channel, Gain)
    """
    if self._channel == self.CHANNEL_A_128:
        return 'A', 128
    if self._channel == self.CHANNEL_A_64:
        return 'A', 64
    if self._channel == self.CHANNEL_B_32:
        return 'B', 32

@channel.setter
def channel(self, value):
    """
    Set input channel
    HX711.CHANNEL_A_128 - Channel A with gain 128
    HX711.CHANNEL_A_64 - Channel A with gain 64
    HX711.CHANNEL_B_32 - Channel B with gain 32
    """
    if value not in (self.CHANNEL_A_128, self.CHANNEL_A_64, self.CHANNEL_B_32):
        raise InvalidMode('Gain should be one of HX711.CHANNEL_A_128,
HX711.CHANNEL_A_64, HX711.CHANNEL_B_32')
    else:
        self._channel = value

    if not self.is_ready():
        self._wait()

    for i in range(self.DATA_BITS):
        self.pd_sck_pin.value(1)
        self.pd_sck_pin.value(0)

    self._set_channel()

def is_ready(self) -> bool:
    """
    When output data is not ready for retrieval,

```

digital output pin DOUT is high.

"""

return self.d_out_pin.value() == 0

def power_off(self):

"""

When PD_SCK pin changes from low to high
and stays at high for longer than 60 us ,
HX711 enters power down mode.

"""

self.pd_sck_pin.value(0)

self.pd_sck_pin.value(1)

sleep_us(self.SLEEP_DELAY_USEC)

def power_on(self):

"""

When PD_SCK returns to low, HX711 will reset
and enter normal operation mode.

"""

self.pd_sck_pin.value(0)

self.channel = self._channel

def read(self, raw=False):

"""

Read current value for current channel with current gain.
if raw is True, the HX711 output will not be converted
from two's complement format.

"""

if not self.is_ready():

self._wait()

raw_data = 0

for i in range(self.DATA_BITS):

self.pd_sck_pin.value(1)

self.pd_sck_pin.value(0)

raw_data = raw_data << 1 | self.d_out_pin.value()

self._set_channel()


```
if raw:
    return raw_data
else:
    return self._convert_from_twos_complement(raw_data)
```

ServoESPHX711.py

```

from machine import Pin, PWM
from machine import I2C
from binascii import hexlify
import time
import umail
import math
from hx711_gpio import HX711
import espnow

sender_email = 'anwarfanclub@gmail.com'
sender_name = 'ESP32 1'
sender_app_password = 'yioy pvwt rsnd ejkg'
recipient_email = 'anwarfanclub@gmail.com'
email_subject = 'PARCEL Movement Detected: Accelerometer'

#sta = network.WLAN(network.STA_IF)
#sta.active(True)
#sta.disconnect()
sg90 = PWM(Pin(22, mode=Pin.OUT))
sg90.freq(50)

e = espnow.ESPNow()
e.active(True)

pin_OUT = Pin(32, Pin.IN, pull=Pin.PULL_DOWN)
pin_SCK = Pin(14, Pin.OUT)

hx711 = HX711(pin_SCK, pin_OUT)
hx711.set_gain(128)

b = hx711.read_average(times=50)
print(b)
time.sleep(2)

led_12 = Pin(12, Pin.OUT)

```

```

parcel = 0

#indicates ready to calibrate to yerba
led_12.value(1)
time.sleep(2)

print("taring")
m = (488)/(hx711.read_average(times=15)-b) #grams per raw unit
print(m)
led_12.value(0)
#scale calibrated

while True:
    host, message = e.irecv(1000)
    if message:
        print("hi")
        message_str = message.decode('utf-8')
        print(message_str)
        if message_str == "unlock":
            print("hi, im unlocking")
            sg90.duty(20)
            time.sleep(1)
        if message_str == "lock":
            print("hi, i'm locking")
            sg90.duty(60)
            time.sleep(1)
        #print("loop")
        time.sleep(.1)

    weight = m*(hx711.read_average(times=3)-b)
    print(round(weight, 1))

    if weight > 150 and parcel == 0:
        print("parcel detected")
        parcel = 1
        smtp = uamil.SMTP('smtp.gmail.com', 465, ssl=True)

```

```
smtp.login(sender_email, sender_app_password)
smtp.to(recipient_email)
smtp.write("From:" + sender_name + "<" + sender_email + ">\n")
smtp.write("Subject:" + "PARCEL Package Arrival Detected: Load Cell" + "\n")
smtp.write(f"A package has been delivered.\n\nWeight(g): {weight:.2f}")
smtp.send()
smtp.quit()
time.sleep(3)
elif weight < 150 and parcel == 1:
    parcel = 0
    print("parcel removed")
    smtp = uemail.SMTP('smtp.gmail.com', 465, ssl=True)
    smtp.login(sender_email, sender_app_password)
    smtp.to(recipient_email)
    smtp.write("From:" + sender_name + "<" + sender_email + ">\n")
    smtp.write("Subject:" + "PARCEL Package Removal Detected: Load Cell" + "\n")
    smtp.write(f"A package has been removed.")
    smtp.send()
    smtp.quit()
```

05/10

ANWAR FANCLUB

ESP 2 IMAGES

Iteration 1, 2, and 3 of ESP32 2

Screenshot image of Email Functionality

PARCEL Movement Detected: IR Inbox x

A

ESP32 <anwarfanclub@gmail.com>
to bcc: me ▾

Movement has been detected near your package by the infrared sensor.

↩ Reply

➡ Forward

ESP32 2 CODE (Typed Out):

ESP32 3 CODE (Typed Out):

```

import network
import espnow
from machine import Pin, PWM, Timer
import time

push_button = Pin(14, Pin.IN, Pin.PULL_DOWN)

buzzer = Pin(27, mode=Pin.OUT)
duty_cycle = 100

sta = network.WLAN(network.STA_IF)
sta.active(True)
sta.disconnect()
e = espnow.ESPNow()
e.active(True)

peer = b'\x14\x2b\x2f\xaf\x62\xc8' # MAC address of peer's wifi interface
e.add_peer(peer) # Must add_peer() before send()
press = 0

while True:
    host, msg = e.irecv(1000) # Wait for message with a timeout of 1000ms
    if msg: # msg == None if timeout in recv()
        print("hi")
        message_str = msg.decode('utf-8')
        print(message_str)
        if message_str == "Movement Detected":
            for i in range(3):
                L1 = PWM(buzzer,freq=450,duty=duty_cycle)
                time.sleep(.2)
                L1 = PWM(buzzer,freq=850,duty=duty_cycle)
                time.sleep(.2)
                L1 = PWM(buzzer,freq=650,duty=duty_cycle)
                time.sleep(.4)
                L1 = PWM(buzzer,freq=1,duty=duty_cycle)
            if message_str == "Acceleration Detected":

```

```

for i in range(1):
    L1 = PWM(buzzer,freq=659,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=622,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=659,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=622,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=659,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=494,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=587,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=523,duty=duty_cycle)
    time.sleep(0.2)
    L1 = PWM(buzzer,freq=440,duty=duty_cycle)
    time.sleep(0.4)
    L1 = PWM(buzzer,freq=1,duty=duty_cycle)
logic_state = push_button.value()
print(logic_state)
print(press)
if logic_state == True and press == 0:
    message = "lock"
    print(message)
    e.send(peer, message)
    press = 1
elif logic_state == True and press == 1:
    message = "unlock"
    e.send(peer, message)
    print(message)
    press = 0
print("looping")

```